Assignment 3: Software-Defined Networking

COL7524: Advanced Networks
Deepak - 2025CSY7616
Pratik - 2025MCS2096

Experiments run locally on Linux with Python 3.12.3, Mininet, Open vSwitch 3.3.9, OpenFlow 1.3, Ryu 4.34 and iperf 2.1.9. All numbers below are observed results from the included experiment script. Single-run measurements illustrate the required comparisons; they are not statistical estimates.

Part 1: Reactive and proactive forwarding

The supplied two-switch topology is unchanged: h1-h3 attach to s1, h4-h5 to s2. The inter-switch link is 10 Mbps with 5 ms delay. Both existing controllers are preserved. Reactive forwarding learns source MAC ports, sends unmatched packets to Ryu, floods unknown destinations and installs rules when the destination is known. Proactive forwarding installs five destination-MAC rules per switch and one broadcast rule before host traffic starts. Fixed host MACs come from autoSetMacs=True.

Each fresh run starts the controller and network, saves initial tables, runs pingAll, sends ten h1-to-h5 pings, and saves final tables. Both runs had 0% pingAll loss. Packet-In count includes the same complete connectivity and ping workload; it includes ARP and other received Ethernet traffic handled by the reactive application.

Measure	Reactive	Proactive
Logged Packet-In events	103	0
h1-h5 mean RTT (ms)	10.140	10.138
Initial rules / switch	1 table miss	6 forwarding rules

Evidence: reactive initial tables contain priority=0 actions=CONTROLLER:65535. Final tables contain learned priority=1 matches such as in_port and dl_dst. Proactive initial tables already contain priority=10 destination matches and priority=5 broadcast flooding, with no controller output rule. Thus proactive traffic requires no Packet-In handling; its zero is supported by the tables, not merely the absence of logging.

```
Reactive:  priority=0 actions=CONTROLLER:65535
Proactive: priority=10,dl_dst=00:00:00:00:00:05 actions=output:4
           (on s1; s2 outputs to port 2)
```

Reactive rules follow observed demand but the first packets incur controller work and setup delay. Proactive rules avoid this setup delay at the cost of advance topology knowledge and rules for destinations that might never be used. Warmed-up ping RTT is nearly identical because both then forward in the switches. This test does not measure first-packet delay separately.

Part 2: Link-metric-based routing

p2/routing.py represents the supplied four-switch ring as a NetworkX graph. Hosts h1-h4 have known MACs and attach to port 1 of s1-s4 respectively. The graph and port map are fixed assumptions, not discovered dynamically. Link delay in milliseconds is the sole static metric. Dijkstra shortest-path selection minimizes summed delay, then installs destination/source MAC and EtherType rules on every switch in that direction. Replies independently install a reverse path. Idle timeout is five seconds.

Link	Mbps	Delay / cost (ms)	Ports
s1-s2	2	20	s1:2, s2:2
s2-s3	2	10	s2:3, s3:2
s3-s4	1	20	s3:3, s4:2
s4-s1	1	15	s4:3, s1:3

ARP broadcasts are delivered by the controller directly to the other host attachment ports. They are never flooded around the ring, preventing a broadcast loop. Unicast ARP and IPv4 use the chosen path. Non-host sources and unknown unicast destinations are ignored. This is sufficient for the supplied fixed IPv4 network.

Baseline h1-to-h3 costs are 20+10=30 ms through s2 and 15+20=35 ms through s4. The selected path is [1, 2, 3]; replies select [3, 2, 1]. In the second test DELAY12=40 changes the controller cost only, leaving the physical link unchanged. The upper path now costs 50 ms and the lower path 35 ms; logs show [1, 4, 3] and [3, 4, 1]. Changing a cost is explicitly the tested operation, not a physical link modification.

Test	Selected path	Mean RTT (ms)
Baseline	s1-s2-s3	60.899
Cost12 = 40	s1-s4-s3	70.914

Both tests had 0% pingAll loss. Approximate expected RTTs are 60 ms and 70 ms, twice the physical path delay; processing and scheduling add overhead. Final s1 IPv4 h1-to-h3 rules output to port 2 in the baseline and port 3 with changed cost. Full tables and PATH log lines are retained in results/p2-*.

Part 3: Congestion-aware routing

Part 3 uses the same controller with CONGESTION=1. Every second it requests port statistics. For each inter-switch port it estimates utilization as 8 times the change in transmitted bytes divided by elapsed seconds and link capacity in bits/s. The undirected link uses the maximum of its two directional estimates. If utilization exceeds 0.70, its delay cost receives a 100 ms penalty. New host-pair/EtherType traffic uses the updated graph. Existing rules continue until five idle seconds; active flows are not rerouted.

For each static/adaptive run, pingAll checks connectivity, then six idle seconds let warm-up rules expire. A 24-second UDP background flow h1-to-h2 offers 1.8 Mbps on the 2 Mbps s1-s2 link. After five seconds for statistics, a new h1-to-h3 ping is measured, followed by an eight-second 0.6 Mbps UDP foreground flow. The two UDP flows compete for the upper 2 Mbps link under static routing. The adaptive foreground path can use the separate 1 Mbps lower route. The pair-based rule granularity means the new UDP flow uses the path selected by the immediately preceding h1-to-h3 ping.

Mode	Mean ping RTT (ms)	Receiver Mbps, first 7 s
Static	61.965	0.480
Adaptive	71.495	0.617

Receiver throughput is the mean of the first seven complete one-second iperf server intervals, excluding end-of-test summary/ACK behavior. Raw interval rates and loss counts remain in the logs. Both modes passed pingAll with 0% loss.

Selected-path evidence (last h1-to-h3 decision in each run):

```
p3-static: PATH 1 -> 3: [1, 2, 3] adaptive=False
```

Measured s1 port 2 utilization at that decision: 1.0.

```
p3-adaptive: PATH 1 -> 3: [1, 4, 3] adaptive=True
```

Measured s1 port 2 utilization at that decision: 0.97.

With the penalty the upper route costs at least 130 ms, so the 35 ms lower route is preferred. Compare the receiver UDP rates above: avoiding the loaded upper link improves the foreground delivery in this scenario, while the alternate route has a longer propagation delay. Queueing under static routing can make its observed ping delay much larger than the unloaded 60 ms RTT.

Limitations: one-second samples react with delay and can miss bursts; no hysteresis is used, so choices near the threshold may oscillate. Fixed host-pair rules do not distinguish TCP/UDP sessions, and active sessions are not moved. Both routing modes use the same small controller and test workload.

Reproduction and evidence

The archive contains unchanged p1_topo.py and p3_topo.py, both Part 1 controllers, the shared Part 2/3 controller, the experiment driver, raw logs/tables and this report. No topology changes were made. The original p2/graph.py is retained as a standalone illustration, not used for packet forwarding.

From the extracted code-a3 folder, install and run:

```
sudo apt install mininet openvswitch-switch iperf python3-venv
python3 -m venv .venv
.venv/bin/pip install setuptools==67.6.1 wheel
.venv/bin/pip install -r requirements.txt
sudo systemctl start openvswitch-switch
sudo python3 experiments/run.py "$PWD/.venv/bin/ryu-manager"
```

The driver runs all six experiments sequentially and overwrites results. Mininet uses system Python; the controller uses the virtual environment. The controller listens at 127.0.0.1:6633. See SETUP.txt for individual interactive commands. Stop other controllers first; sudo mn -c cleans up after interruption.

For interactive Part 3, start CONGESTION=1 with p2/routing.py and start p3_topo.py in a second terminal. The p3 folder documents the shared implementation; a duplicate controller is unnecessary.

```
CONGESTION=1 .venv/bin/ryu-manager \
  --ofp-tcp-listen-port 6633 p2/routing.py
sudo python3 p3_topo.py
# Mininet examples:
pingall
h1 ping -c 10 h3
sh ovs-ofctl -O OpenFlow13 dump-flows s1
```

Evidence files: p1-reactive/proactive.txt contain initial/final rules, ping results and counts. p2-base/changed.txt and their controller logs contain cost-change results and selected paths. p3-static/adaptive.txt contain loaded ping, UDP client/server output and flow tables; their controller logs include measured utilization and paths.

Ryu is third-party software. The bundled Apache-2.0 wheel includes the existing eventlet ALREADY_HANDLED compatibility adjustment from this environment. It is included to remove an external local-source dependency. NetworkX, Ryu and Mininet perform standard graph, OpenFlow and emulation tasks; the submitted controllers implement the assignment policy.

Viva outline: explain table-miss versus preinstalled rules; add link costs to justify each route; derive utilization from byte counters; explain the threshold penalty, idle timeout, and why the existing background flow stays on its original route.